ChatGPT

DocuMint: Blockchain-Enabled DigiLocker for Hackathons

- **What & Why:** A friendly, NFT-powered "DigiLocker" for official documents – say goodbye to paper hassles and fraud. Users mint their documents as NFTs (each with an IPFS hash) on the blockchain. This makes them **immutable, verifiable and ultra-secure** [1] [2]. Once on-chain, docs are tamper-proof and instantly retrievable by anyone with permission [1]. No more lengthy verifications – it's **lightning-fast and fraud-free** (simply scan a QR code to verify authenticity, any mismatch flags fraud immediately [3]).

- **Key Features:**

- **NFT Document Minting:** Turn each certificate or ID into an ERC-721 NFT; it's unique and publicly verifiable [4]. Once minted, the credential *cannot be altered or forged* [4] [5].
- **Decentralized Storage:** Files live on IPFS (via Pinata). Only the cryptographic hash is saved on-chain [2] [6]. IPFS ensures **decentralized, content-addressed storage** – data can't be changed without changing the hash [7] [6].
- **QR-Code Access:** Each NFT comes with a QR code. Scanning it immediately pulls up the IPFS document and checks the on-chain hash for a match [3]. If the hashes match, authenticity is confirmed in seconds – flagging any tampered or fraudulent docs quickly [3].
- **Verification Portal:** A web portal lets verifiers (e.g. HR, border control, university staff) scan QR codes or enter an ID to instantly fetch and verify documents against the blockchain record (no middleman needed, no waiting).
- **Admin Panel & Access Control:** Admins can authorize entities (banks, government agencies, university depts) to mint/verify docs. Access control is enforced by smart contracts (only authorized roles can mint or mark valid) and by encrypted QR permissions (role-based QR codes).
- **Encryption & Privacy:** Documents are AES-encrypted before IPFS upload [6], so only holders with the decryption key (e.g. rightful owner or verifier) can read them. Sensitive personal data stays private even on a public network.
- **Anti-Fraud:** Immutable blockchain + unique NFTs = **no backdating or fake certificates** [4] [5]. QR codes and cryptographic checks catch any alterations immediately. Verifiers instantly know if a document is genuine or spoofed.

Flow: Minting a document to NFT. User uploads & (optionally encrypts) a document → it's pinned to IPFS (gets a unique CID) → smart contract mints an ERC-721 NFT with that CID in its metadata [6] [2]. The new NFT/QR is tied to the owner's wallet. The IPFS hash on-chain makes the doc immutable and verifiable.

Flow: Verifying via QR. Verifier scans the document's QR code → app fetches the IPFS content and contract data → it compares the document's hash to the hash stored on-chain [3] [2]. If they match, the doc is authentic (all in one quick step). Any mismatch signals potential fraud immediately [3] [2].

Tech Stack & Architecture

- **Frontend:** React.js + Web3 for dApp UI (users can connect MetaMask to upload/verify docs). Fast, responsive interfaces for mobile and web.
- **Backend:** Node.js/Express server (optional) for handling file uploads, interacting with IPFS (via Pinata API), and serving the verification portal.
- **Blockchain:** Ethereum (or any EVM chain) smart contract in Solidity (ERC-721 for NFT docs). Contracts handle minting, role-based access, and store document hashes. We use OpenZeppelin libraries for security.
- **Storage:** IPFS (via Pinata or Infura) for content-addressed file storage. Only the IPFS CID (hash) is kept on-chain [6]. This offloads heavy file data from the blockchain while preserving integrity.
- **Wallet/Keys:** MetaMask or any Web3 wallet for users and admins to sign transactions. Role-based permission keys built into smart contracts prevent unauthorized minting or admin actions.
- **Encryption:** AES or similar symmetric encryption on the client side before upload [6]. This ensures privacy: only holders with the key (e.g. after scanning a QR code) can decrypt the file.
- **QR Integration:** A QR-code library generates a code linking to the doc's verification URL (or CID). Scanning triggers the React app to fetch data and run the verify logic.
- **APIs & Tools:** Pinata/IPFS API (for pinning/fetching docs); Ethers.js/web3.js (blockchain calls); React Router; Node.js server for QR generation and serving verification; any database (optional) for mapping users to wallets or storing access logs.

Modules & Components

- **User Module:** Uploads document, enters metadata, triggers `mintNFT()` contract call (owner/admin only) which mints the NFT and generates QR.
- **Verifier Module:** Scans QR or enters ID → frontend calls a backend or contract directly to fetch IPFS hash from the contract and IPFS file content → compares hashes. Shows "✔ Valid" or " Invalid" immediately.
- **Admin Panel:** Interface for admins to manage roles (e.g. add issuer or verifier addresses), view stats (how many docs minted, flagged), and pull unverified documents.
- **Smart Contracts:** ERC-721 contract with an `onlyAdmin` mint function. It stores tokenURI = IPFS URL of encrypted document. Access control modifiers ensure only authorized roles can mint or invalidate tokens.
- **Infrastructure:** Ethereum network (Mainnet or testnet for hackathon), IPFS nodes, and optionally a simple backend server (Node.js) to glue IPFS and contracts and to host the verification portal.

Sample Smart Contract (Solidity)

// SPDX-License-Identifier: MIT

```
pragma solidity ^0.8.0;
import "@openzeppelin/contracts/token/ERC721/ERC721.sol";
contract DocNFT is ERC721 {
    uint public nextId;
    address public admin;
    constructor() ERC721("DocNFT","DOC") { admin = msg.sender; }
```

```
    function mintDocument(address recipient, string calldata tokenURI) external
{
        require(msg.sender == admin, "Not authorized");
        _safeMint(recipient, nextId);
        _setTokenURI(nextId, tokenURI);
        nextId++;
    }
}
```

Simple ERC-721 NFT contract: only `admin` *can mint, incrementing* `nextId` *per doc. The* `tokenURI` *holds the IPFS link (encrypted doc). Easy to read and modify for role-based extensions.*

Real-World Use Cases

- **Government IDs / KYC:** Issue blockchain-backed IDs or certificates (like Aadhaar, driving licenses) that employers and banks can instantly verify [8]. Eliminates redundant checks and cuts fraud (government agencies become authorities that sign these NFTs).
- **University Credentials:** Diplomas, transcripts as NFT certificates [4]. Students get an immutably signed record of their degree. Employers and schools globally can instantly verify without calling the registrar. (Reduces counterfeit degree fraud).
- **Immigration / Travel:** Visas and passports with unique QR codes on blockchain. Border agents scan and verify instantly that the document is issued by the state and unaltered. No more fake passports or stolen visa fraud.
- **Corporate KYC & Records:** Banks or companies can store customer KYC docs (e.g. business licenses) as verifiable NFTs. Onboarding becomes faster and more secure – banks simply scan and trust the blockchain proof instead of paper.
- **Healthcare Credentials:** Medical licenses, patient consents, or lab reports stored immutably. Hospitals scan to verify certifications or test results, protecting patient data with encryption while preventing tampering.
- **Supply Chain:** (Bonus) Certificates of origin or quality assurance documents as NFTs. At customs or handoffs, parties scan QR codes to verify origin/authenticity using the blockchain ledger.

All use cases benefit from blockchain's tamper-resistance [1] [5] and privacy controls. For example, Dock Labs notes blockchain IDs boost security by eliminating centralized third parties and preventing fraud [8], which directly applies here (only rightful users control their data).

Hackathon Pitch Slide

DocuMint – "Your Docs on the Blockchain" – *"Instantly issue and verify anything from diplomas to driver's licenses as NFT credentials. No paperwork, no fraud – just scan and trust!"*

One-liner: *"Secure your documents as NFTs on a tamper-proof blockchain, with instant QR-code verification!"*

Why We're Awesome: Immutable IDs with ultra-fast verification (thanks to blockchain+IPFS) [2] [3]; privacy by design with encryption [6]; hackathon-ready with React/Node/MetaMask stack; real impact in KYC, universities, government – a fully decentralized DigiLocker.

1 GitHub - DevAloshe/BlockChain-Based-Document-Verfication-With-IPFS
https://github.com/DevAloshe/BlockChain-Based-Document-Verfication-With-IPFS

2 7 Blockchain IPFS: Ultimate Guide to Decentralized Storage |2024
https://www.rapidinnovation.io/post/blockchain-ipfs-comprehensive-guide-to-decentralized-storage-solutions

3 How to Use QR Codes for Document Verification?
https://qrcodedynamic.com/blog/qr-code-for-document-verification/

4 NFTs Revolutionizing Academic Credential Verification
https://www.rapidinnovation.io/post/how-nfts-revolutionize-academic-credential-verification

5 8 Blockchain Identity Management: Beginner's Guide 2025
https://www.dock.io/post/blockchain-identity-management

6 Design a Document Verification System Based on Blockchain Technology
https://www.atlantis-press.com/article/125979641.pdf